



Universidad
Rey Juan Carlos

HIGHER TECHNICAL SCHOOL OF COMPUTER ENGINEERING
MASTER'S DEGREE IN COMPUTER GRAPHICS, GAMES, AND VIRTUAL
REALITY

2025-2026 Academic Year

Advanced Rendering I

Practice 2

Carlos Escribano González

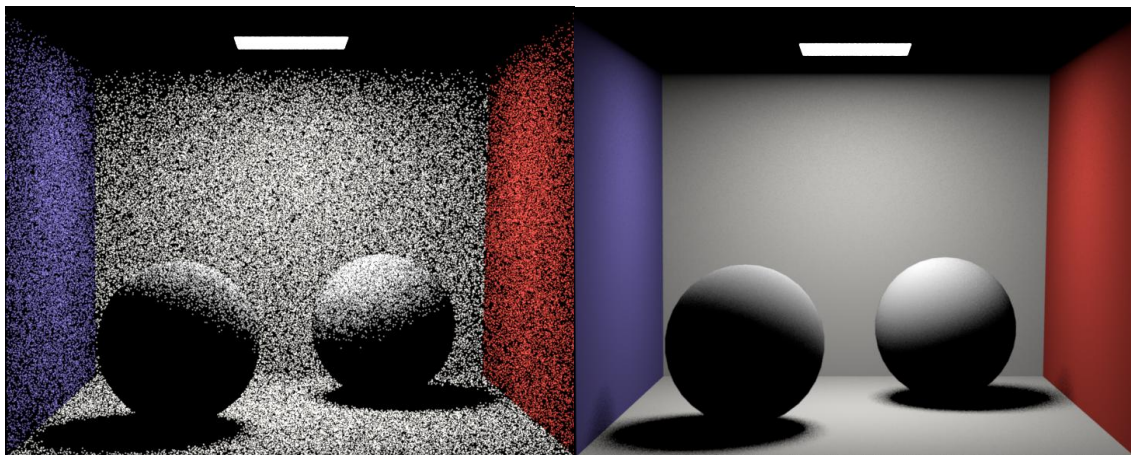
Area Light Sampling:

For area light sampling, the *sampleLight()* function was implemented in *direct.cpp*. This function stores, in a *SampledInfo* struct, the sample taken from a scene light, the corresponding evaluation of the material's BSDF, and the PDF of the selected emitter, which will be used later for Multiple Importance Sampling.

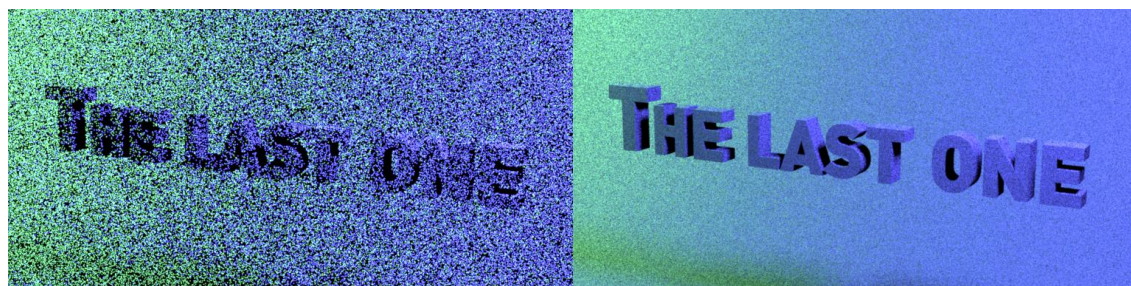
The light to be sampled is selected using the *sampleEmitter()* function from *scene.cpp*, which randomly selects an emitter, initially with uniform probability. A sample is then drawn from it. The point on the light is selected using the *samplePoint()* function from *mesh.cpp*, which selects a random triangle weighted by its area and, using *squareToTent()*, samples a point within its domain. Finally, the *pdf()* function in *area.cpp* converts the area-based PDF into a solid-angle PDF.

Therefore, a single call to *sampleEmitter()* from *direct.cpp* provides the evaluation of a sample on a scene light, already divided by its solid-angle PDF. After evaluating the material's BSDF and including the geometry term, the only remaining step is to check its visibility. If a ray is cast towards the light with a maximum distance slightly smaller than the distance between the evaluated point and the emitter, and it intersects any object in the scene, black is returned.

The results obtained using 32 samples per pixel are shown below:



Material sampling (left) vs. light sampling (right); 32 spp

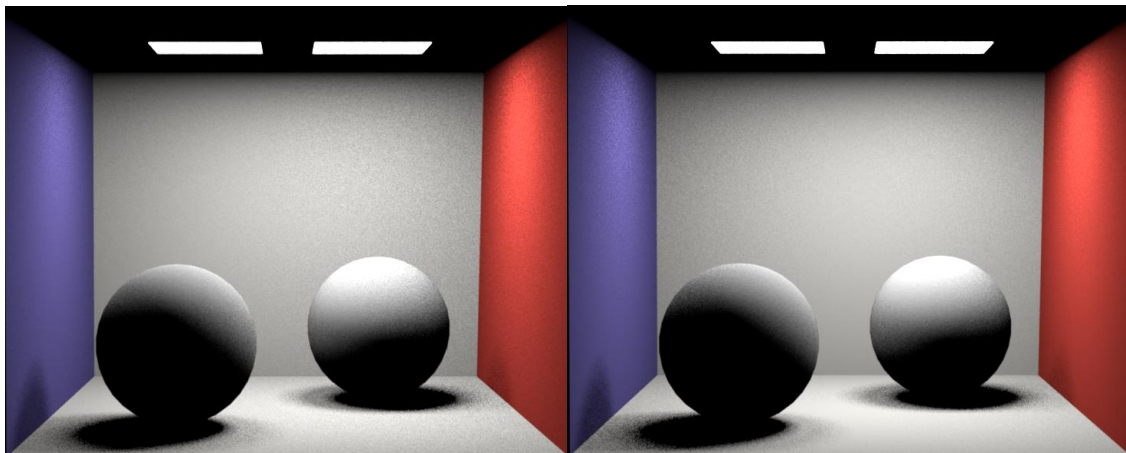


Material sampling (left) vs. light sampling (right); 32 spp

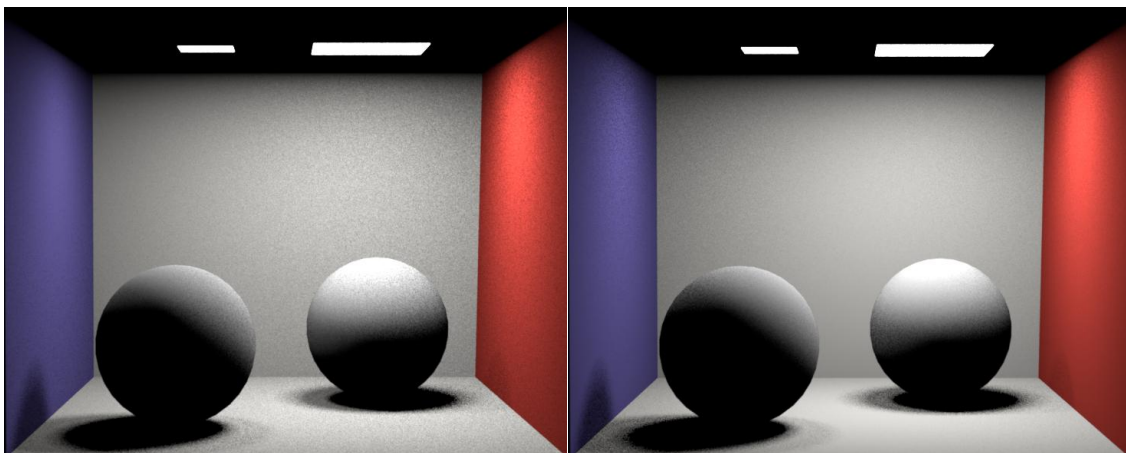
It can be observed that light sampling provides noticeably better results than material sampling for Lambertian materials under direct lighting. This is expected, since the probability that a sample drawn according to the material's BSDF will land on an emitter is low. In this case, the resulting samples are not very representative of the integrand, requiring more iterations to reduce the variance of the estimator. Material sampling performs better for materials with a stronger specular component.

Emitter Sampling Based on Radiance

To sample emitters with a probability associated with their radiance, the *DiscretePDF* structure was used. In the *activate()* function in *scene.cpp*, the structure is initialized by adding one value for each emitter in the scene. This value is equal to the emitter's total area multiplied by its luminance, thereby assigning greater weight to lights that have a greater influence on the scene's illumination. Other approaches were also considered, such as using the norm of the radiance instead of its luminance, or using a probability based solely on area. After modifying the *sampleEmitter()* function to use the previously mentioned *DiscretePDF* structure, the following results were obtained:



Uniform sampling (left) vs. luminance and area-based sampling (right); right light with twice the radiance



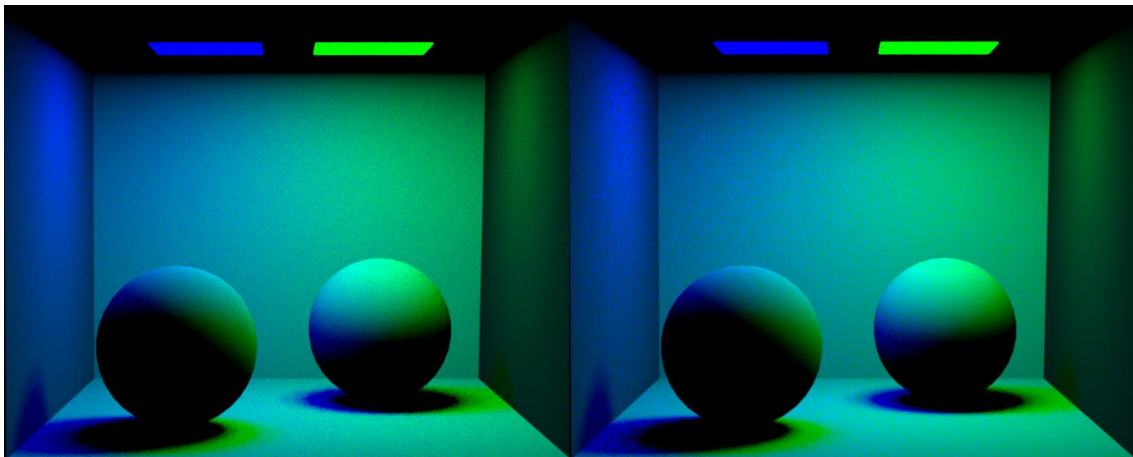
Uniform sampling (left) vs. luminance and area-based sampling (right); lights with different sizes



Uniform sampling (left) vs. luminance and area-based sampling (right)

It can be observed that sampling lights based on their luminance and area produces slightly less noise, as it reduces the variance by obtaining more representative samples more frequently.

As shown in the following example, the result can also vary depending on the type of radiance weighting used. For example, sampling based on luminance, compared with sampling based on the norm of the radiance, gives greater weight to colors perceived as more intense, even when both lights have the same total radiance:

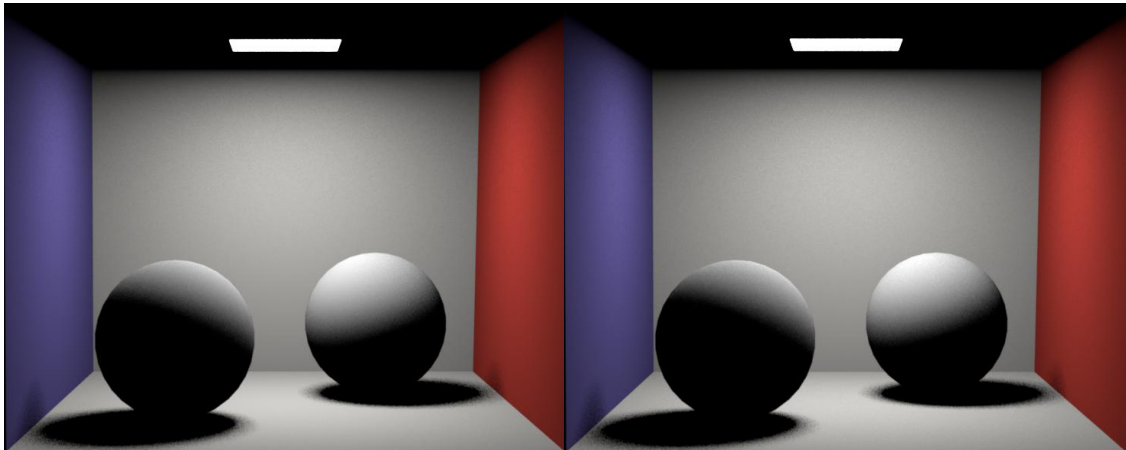


Sampling based on the norm of the radiance (left) vs. luminance-based sampling (right)

Although the noise on the left wall, which is primarily affected by the blue light, increases, luminance-based sampling noticeably reduces the noise in the rest of the image, resulting in a cleaner render.

Multiple Importance Sampling:

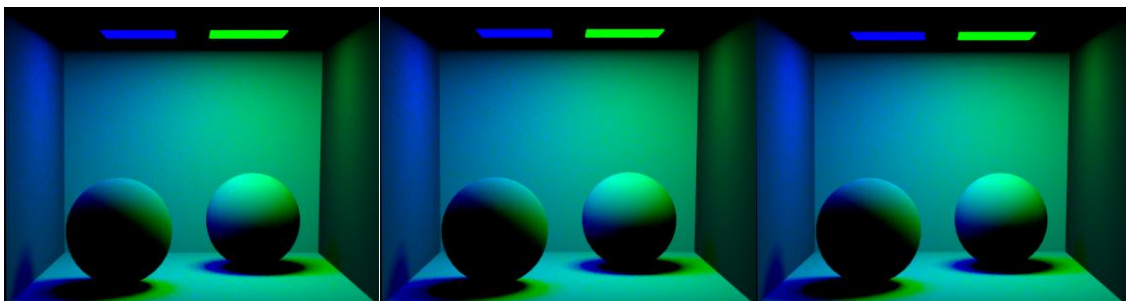
For the implementation of Multiple Importance Sampling, the *MIS()* function was created in *direct.cpp*. It receives the information from material sampling and light sampling, previously stored in *SampledInfo* structures. From these values, it calculates and returns the final color using the balance heuristic, power heuristic, or maximum heuristic. The method is selected using an enumerator of type *MIS_MODE*.



Light sampling (left) vs. MIS with balance heuristic (right)



Light sampling (left) vs. MIS with balance heuristic (right)

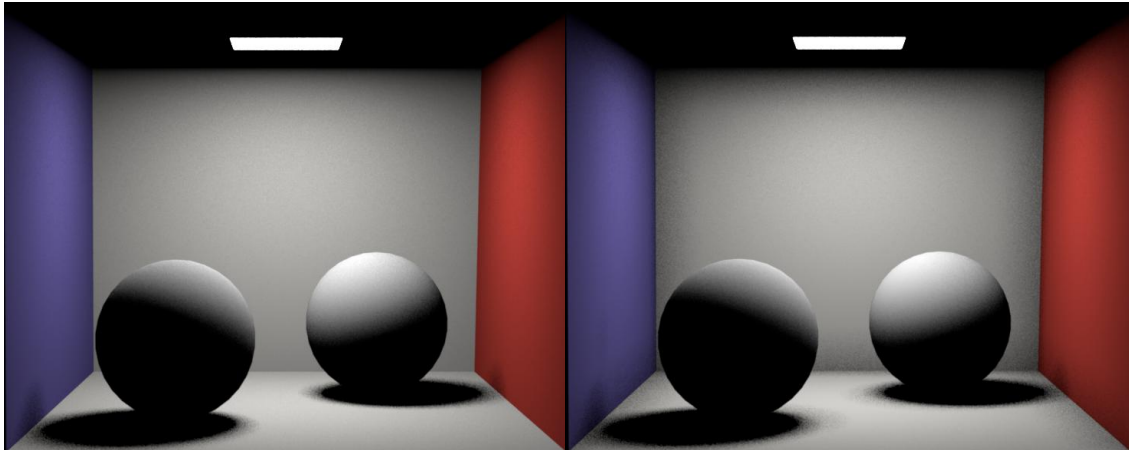


Balance heuristic (left) vs. power heuristic (center) vs. maximum heuristic (right)

For such simple scenes, the difference is barely noticeable.

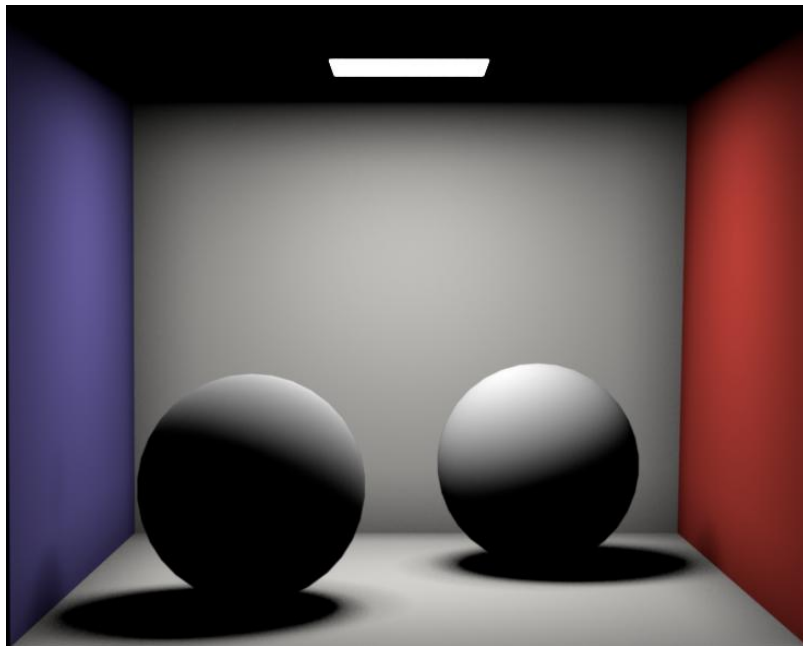
Ambient Occlusion

An additional *ambientOcclusion()* function was added to *direct.cpp*. If a ray cast from the evaluated point, with a predefined maximum distance, intersects any object in the scene, it returns 0; otherwise, it returns 1. This value is multiplied by the color calculated by *Li()*, causing pixels to become darker as they become more occluded.



Scene without ambient occlusion (left) vs. with ambient occlusion (right)

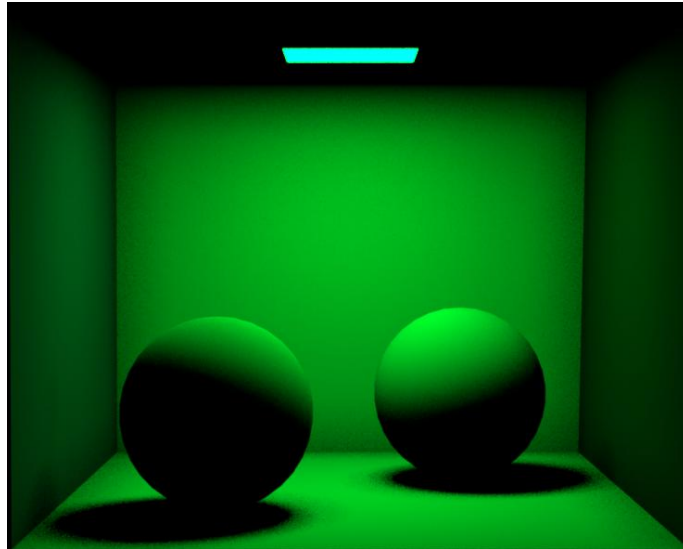
With a larger number of samples per pixel, the ambient occlusion result is as follows:



Ambient occlusion with 512 samples per pixel

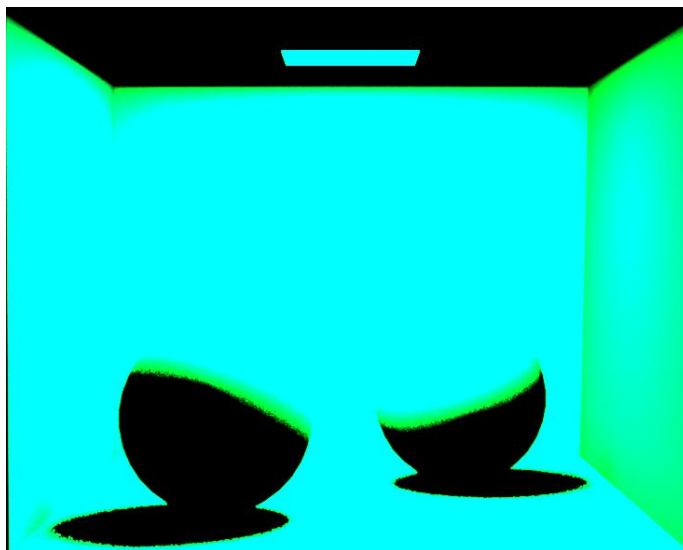
Tone Mapping

After completing the practice, what appeared to be an artifact was identified: for some area lights with non-uniform radiance across their three color channels, the resulting color of the emitter may not match the color of the illumination it produces on the objects in the scene, as shown below:



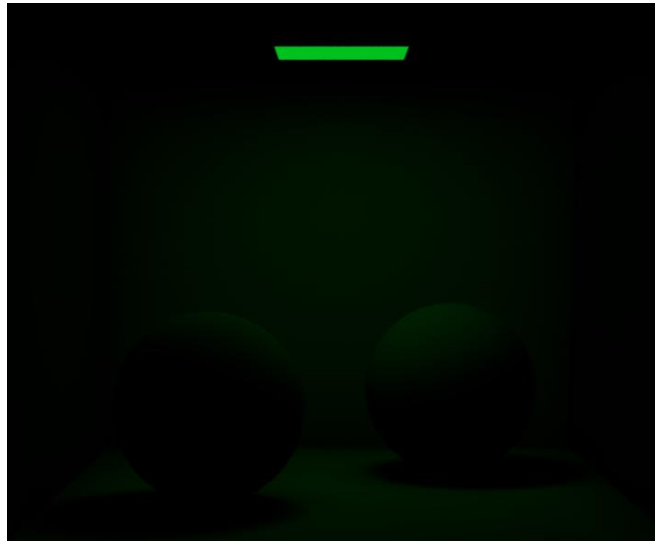
Green illumination, cyan emitter

This scene contains only one area light with a radiance of 40 in the green channel and 1 in the blue channel. However, the emitter appears cyan, which initially seems inconsistent with the green illumination in the scene. The blue component of the light, being much weaker, loses enough energy after the first bounce that it is no longer noticeable in the illumination. However, it is still strong enough to visibly tint the color of the emitter at the default exposure value. The presence of the blue channel in the scene's illumination can also be verified by increasing the exposure:



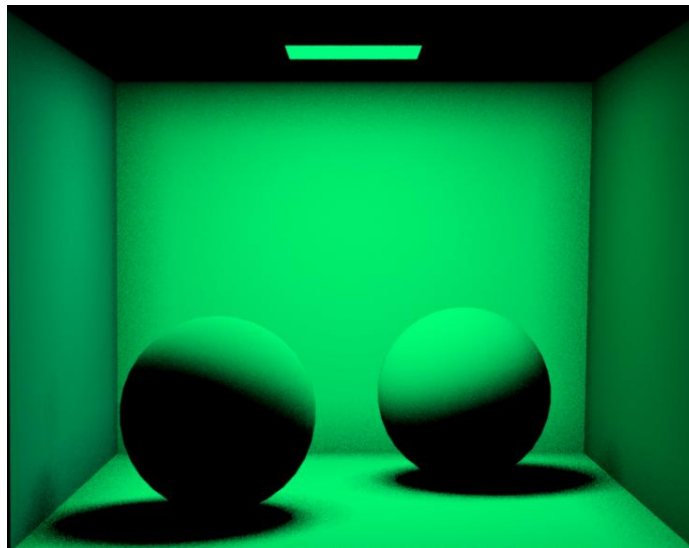
Same scene, exposure increased to the maximum

Conversely, the blue component disappears even from the emitter when the exposure is reduced:



Same scene, reduced exposure

To avoid this visual discrepancy between the color of the emitter and the color of the illumination it produces on the objects in the scene, tone mapping can be implemented. In this case, tone mapping was implemented using the ACES technique and based on luminance. Through the new *applyTonemapping()* function, this effect is applied to the resulting color of each pixel calculated by *Li()*, producing the following result (after also applying gamma correction):



Same scene with tone mapping